

The LQD Model

A log-quantile-density parametrization for arbitrage-free equity smiles

Vol-Fitter Technical Notes, No. 01

Vol-Fitter

July 3, 2026

Abstract

This note develops the slice and surface parametrization that the Vol-Fitter calls *LQD* — “log quantile density”. For a single expiry let $X = \log(S_T/F_T)$ be the log-forward return under the T -forward measure, let $Q(u)$ be its quantile function and $q(u) = Q'(u)$ its quantile density. The model parametrizes

$$\ell(u) = \log q(u) = -\log u - \log(1-u) + (1-u)L + uR + \sum_{n=2}^N a_n P_n(1-2u), \quad u \in (0,1),$$

where the P_n are Legendre polynomials. The smallest production specification $N = 6$ has seven parameters. Because $q = e^\ell > 0$ automatically, the recovered risk-neutral density is non-negative *by construction*: a one-expiry LQD slice is free of butterfly arbitrage after a single scalar martingale normalization, with no positivity penalty anywhere in the optimizer. The logarithmic endpoint terms give exponential return tails, hence linear implied-variance wings that sit automatically inside Lee’s moment bounds; one explicit integrability condition $A_R < 1$ guarantees a finite forward. We give the full pricing equations, the tail and Lee-slope analysis, the exact analytic ATM level/skew/curvature handles, the trader-facing ATM-orthogonal chart, and the calibration objective with its soft tail barrier. A dedicated section derives the model’s *analytic residual Jacobian* — a clean cancellation identity that removes the finite-difference quadrature rebuild and is, to our knowledge, the original engineering contribution of this implementation. Every figure and table is generated fresh by the production calibrator: the seven-parameter fit to an SPX-like SVI-JW smile reaches a maximum implied-volatility error of 0.23 volatility basis points, and a thirteen-parameter fit reproduces a bimodal “double-hat” event density.

Contents

1	Purpose and the modelling choice	3
2	Static arbitrage and the quantile construction	4
2.1	Normalized prices and Breeden–Litzenberger	4
2.2	The logit coordinate and stable integration	5
3	Basis functions and exact coefficients	6
3.1	The Legendre body	6
3.2	Endpoint scales	6

4	Tail asymptotics and Lee moment boundaries	7
4.1	Return moments	7
4.2	Lee wing slopes	7
5	Option pricing and implied-volatility recovery	7
5.1	The Black map and vega-normalized residuals	7
5.2	Quadrature, Hermite interpolation, and Newton inversion	8
6	Exact ATM handles and the orthogonal chart	8
6.1	Level, skew and curvature from the quantile	8
6.2	The ATM-orthogonal chart	9
7	Calibration of one expiry	10
7.1	The objective	10
7.2	Initialization	10
7.3	The hooks: bands, var-swaps, and priors	10
7.4	The var-swap closed form	11
8	The analytic residual Jacobian	11
8.1	The cancellation identity	11
8.2	Nodal sensitivities	12
8.3	Scope and measured speed-up	12
9	Calendar-arbitrage-free surfaces	13
10	Worked examples	13
10.1	An SPX-like SVI-JW smile	13
10.2	A bimodal “double-hat” event smile	14
11	What is genuinely original here	15
12	Limitations	15
13	Traceability	16
A	Hyperparameter atlas	17
B	Performance notes	18
C	The SVI-JW conversion used in the example	18
D	Reference implementation	19

1 Purpose and the modelling choice

Most smile models parametrize implied volatility, total variance, or option price *directly*. That is convenient to quote but awkward to keep arbitrage-free: non-negativity of the implied risk-neutral density is then a *global nonlinear* condition on the curve, policed during optimization by penalties that the fitter can violate on the way to a solution. The LQD model inverts the construction. It parametrizes a probability law first — through the quantile density of the terminal return — and reads option prices and implied volatilities off that law by integration afterwards. The design objective is a single sentence:

Heuristic

A calibrated slice should be a smooth, flexible smile, yet *every* admissible parameter vector should already correspond to a genuine probability law for the terminal forward return. “Fit a curve, then check it is a density” becomes “fit *within* the space of densities, then read off implied volatility.”

Invariant

1. Every parameter vector prices from a genuine non-negative density: butterfly-freedom is structural, never a penalty (proposition 1).
2. The only hard admissibility condition is $A_R < 1$ (a finite forward), guarded by a smooth barrier before the wall.
3. The martingale normalization is a single scalar shift: it changes neither the density’s shape, its tails, nor its positivity.
4. The ATM handles $(\sigma_0, s_0, \kappa_0)$ are exact closed-form functionals of the slice — the same three coordinates the graph extrapolator of Note 14 propagates.

The modelling object is the log-forward return

$$X_T = \log\left(\frac{S_T}{F_T}\right), \quad (1)$$

with F_T the forward for expiry T . We suppress discounting and deterministic carry by working in normalized, undiscounted units; the normalized call at log-moneyness $k = \log(K/F_T)$ is

$$C_T(k) = \mathbb{E}^T \left[(e^{X_T} - e^k)^+ \right], \quad \mathbb{E}^T [e^{X_T}] = 1, \quad (2)$$

the second equality being the martingale (forward) condition. Unless a maturity is displayed, every formula below is for one expiry.

If F_X is the law of X and $Q(u) = F_X^{-1}(u)$ for $0 < u < 1$, the quantile density is

$$q(u) = Q'(u) = \frac{1}{f_X(Q(u))}. \quad (3)$$

Modelling $\ell = \log q$ forces $q > 0$, hence Q strictly increasing, hence a *bona fide* non-negative density. That is the entire structural reason a one-expiry LQD slice carries no butterfly arbitrage. The

chosen basis is

$$\ell(u) = -\log u - \log(1-u) + (1-u)L + uR + \sum_{n=2}^N a_n P_n(1-2u) \quad (4)$$

and it is deliberately *not* a generic polynomial basis. The two singular terms $-\log u$ and $-\log(1-u)$ manufacture logarithmic quantile tails — equivalently, exponential return tails and therefore the linear implied-total-variance wings that Lee’s formula demands. The endpoint coefficients L, R scale those tails; the Legendre modes a_n shape the body. The minimal production model $N = 6$ carries the seven parameters $(L, R, a_2, a_3, a_4, a_5, a_6)$, and higher N adds body detail without ever touching the no-arbitrage mechanism.

In code the positivity is a single line: with the smooth part $g(u) = (1-u)L + uR + \sum_n a_n P_n(1-2u)$, the return density is $f_X(Q(u)) = 1/q(u) = u(1-u) e^{-g(u)}$, non-negative for *any* coefficients.

Listing 1: The LQD density is structurally non-negative — no constraint, no penalty (distilled from `models/lqd/quadrature.py`).

```

1  def density(u, g):                                     # g(u) = (1-u)*L + u*R + sum_n
    a_n P_n(1-2u)
2      return u * (1 - u) * np.exp(-g)                  # = e^{-ell(u)} >= 0 for ANY g
    => no butterfly

```

Why this is the right trade. A direct-IV model spends optimizer effort *policing* arbitrage; the LQD model spends none, because the feasible set *is* the set of densities. The price is two scalar nonlinearities: a martingale normalization (one log of an integral, section 2.2) and a tail admissibility condition $A_R < 1$ (section 4). Both are cheap, and both are imposed once per residual evaluation rather than at every quote.

2 Static arbitrage and the quantile construction

2.1 Normalized prices and Breeden–Litzenberger

Write $Y = e^X = S_T/F_T$ and let $\tilde{C}(y) = \mathbb{E}[(Y - y)^+]$ be the normalized call as a function of the normalized strike $y = e^k$. A static, butterfly-free slice requires $\tilde{C}$ decreasing and convex in y with $\tilde{C}(0) = 1$ and $\tilde{C}(\infty) = 0$. When Y has a density,

$$\frac{\partial \tilde{C}}{\partial y} = -(1 - F_Y(y)), \quad \frac{\partial^2 \tilde{C}}{\partial y^2} = f_Y(y) \geq 0, \quad (5)$$

which is the Breeden–Litzenberger identity: the second strike derivative of the call *is* the risk-neutral density. The quantile construction supplies such a density directly. Given a strictly increasing Q for X , the quantile of $Y = e^X$ is $Q_Y(u) = e^{Q(u)}$; if $\int_0^1 e^{Q(u)} du = 1$ then $\mathbb{E}[Y] = 1$. For $k \in \mathbb{R}$ let u_k solve $Q(u_k) = k$; then

$$C(k) = \int_{u_k}^1 (e^{Q(u)} - e^k) du, \quad (6)$$

$$P(k) = \int_0^{u_k} (e^k - e^{Q(u)}) du, \quad (7)$$

and subtracting gives put–call parity $C(k) - P(k) = 1 - e^k$.

Proposition 1 (A single LQD slice is butterfly-free). *Suppose $q(u) = e^{\ell(u)} > 0$ on $(0, 1)$ and the martingale integral $\int_0^1 e^{Q(u)} du$ is finite. After the scalar shift of Q that makes this integral equal to one, the prices (6) are generated by a probability law on $Y > 0$ with mean one; the slice is therefore free of butterfly arbitrage.*

Proof. Since $q > 0$, Q is strictly increasing and defines a law for X ; $Y = e^X > 0$; the shift enforces $\mathbb{E}[Y] = 1$. The call is the expectation of $(Y - e^k)^+$ under this law, so monotonicity and convexity in strike follow from (5). No separate positivity inequality is ever checked. $\square$

2.2 The logit coordinate and stable integration

The endpoint singularities in (4) vanish under the logit change of variable

$$z = \log \frac{u}{1-u}, \quad u = \Lambda(z) = \frac{1}{1 + e^{-z}}. \quad (8)$$

Writing the smooth part $g(u) = (1-u)L + uR + \sum_{n \geq 2} a_n P_n(1-2u)$, we have $q(u) = e^{g(u)}/(u(1-u))$ and, crucially,

$$\frac{dQ}{dz} = q(u) u(1-u) = e^{g(\Lambda(z))}. \quad (9)$$

All singularities are gone: the quantile is the integral of a smooth positive function on the line,

$$\overline{Q}(z) = \int_0^z e^{g(\Lambda(s))} ds, \quad (10)$$

and a scalar shift μ enforces the martingale constraint,

$$Q(z) = \mu + \overline{Q}(z), \quad \mu = -\log \left(\int_{-\infty}^{\infty} e^{\overline{Q}(z)} \Lambda(z)(1 - \Lambda(z)) dz \right). \quad (11)$$

This is the *only* normalization; it changes neither q , the tails, nor positivity. Pricing is equally clean. Let z_k solve $Q(z_k) = k$, $u_k = \Lambda(z_k)$, and define the *upper asset-share integral*

$$A(z) = \int_z^{\infty} e^{Q(s)} \Lambda(s)(1 - \Lambda(s)) ds. \quad (12)$$

Since $1 - u_k = \int_{z_k}^{\infty} \Lambda(s)(1 - \Lambda(s)) ds$, the call price is

$$\boxed{C(k) = A(z_k) - e^k(1 - u_k)}. \quad (13)$$

A single quadrature grid yields $Q(z)$ and $A(z)$; every strike is then priced by interpolation. This pricing identity is also the seed of the analytic Jacobian of section 8, because the implicit dependence of z_k on the parameters will cancel exactly.

3 Basis functions and exact coefficients

3.1 The Legendre body

With $x = 1 - 2u \in (-1, 1)$ the Legendre polynomials obey $\int_{-1}^1 P_m P_n dx = \frac{2}{2n+1} \mathbf{1}_{m=n}$, hence

$$\int_0^1 P_m(1-2u)P_n(1-2u)du = \frac{1}{2n+1} \mathbf{1}_{m=n}. \quad (14)$$

This is a well-conditioned global basis for the smooth part of ℓ . The first two degrees are carried by the endpoint terms, since

$$(1-u)L + uR = \frac{L+R}{2} + \frac{L-R}{2}(1-2u), \quad (15)$$

so the seven-parameter $N = 6$ model spans the same polynomial space as $P_0, \dots, P_6$ while preserving the endpoint meaning of the first two coordinates. The body polynomials are built by the stable three-term recursion

$$(n+1)P_{n+1}(x) = (2n+1)xP_n(x) - nP_{n-1}(x), \quad P_0 = 1, \quad P_1 = x. \quad (16)$$

3.2 Endpoint scales

The endpoint values of g set the tail scales. Because $P_n(1) = 1$ and $P_n(-1) = (-1)^n$,

$$A_L := e^{g(0)} = \exp\left(L + \sum_{n \geq 2} a_n\right), \quad (17)$$

$$A_R := e^{g(1)} = \exp\left(R + \sum_{n \geq 2} (-1)^n a_n\right). \quad (18)$$

These are not diagnostics — they are the slopes of the quantile in the logit coordinate, $Q(z) = \mu + A_L z + O(1)$ as $z \rightarrow -\infty$ and $Q(z) = \mu + A_R z + O(1)$ as $z \rightarrow +\infty$. The right-tail integrability condition for a *finite forward* is

$$\boxed{A_R < 1}. \quad (19)$$

Indeed, as $z \rightarrow +\infty$, $\Lambda(z)(1 - \Lambda(z)) \sim e^{-z}$ and $e^{Q(z)} \sim C e^{A_R z}$, so the martingale integrand decays like $e^{(A_R - 1)z}$, integrable iff $A_R < 1$. On the left the integrand decays like $e^{(A_L + 1)z}$ for every $A_L > 0$, so no left condition is needed.

Caution

$A_R < 1$ is a genuine hard constraint, not a regularization preference: an LQD slice with $A_R \geq 1$ has an *infinite* forward and its “prices” are meaningless. The calibrator enforces it with a hard rejection *and* a smooth soft barrier that engages well before the boundary (section 7), so the finite-difference and analytic Jacobians stay smooth as the optimizer approaches the wall.

4 Tail asymptotics and Lee moment boundaries

4.1 Return moments

The endpoint linearity of Q in z gives exponential return tails. For large x , $z \sim x/A_R$ and $1-u \sim e^{-z}$, so

$$\mathbb{Q}(X > x) \asymp e^{-x/A_R}, \quad x \rightarrow +\infty, \quad (20)$$

and symmetrically $\mathbb{Q}(X < x) \asymp e^{x/A_L}$ as $x \rightarrow -\infty$. The moment generating function of X therefore has critical strip $-1/A_L < r < 1/A_R$, and since $Y = e^X$ has mean one,

$$p_R^* = \sup\{p \geq 0 : \mathbb{E}[Y^{1+p}] < \infty\} = \frac{1}{A_R} - 1, \quad q_L^* = \sup\{q \geq 0 : \mathbb{E}[Y^{-q}] < \infty\} = \frac{1}{A_L}. \quad (21)$$

4.2 Lee wing slopes

Let $w(k, T) = \sigma_{BS}^2(k, T) T$ be total implied variance. Lee's moment formula gives the asymptotic total-variance slopes $\beta_R = \limsup_{k \rightarrow +\infty} w/k$ and $\beta_L = \limsup_{k \rightarrow -\infty} w/|k|$ as

$$\beta = \psi(p), \quad \psi(p) = 2 - 4(\sqrt{p^2 + p} - p), \quad p \in [0, \infty], \quad (22)$$

equivalently $\frac{1}{2\beta} + \frac{\beta}{8} - \frac{1}{2} = p$, $0 < \beta \leq 2$. Combining with (21),

$$\beta_L = \psi\left(\frac{1}{A_L}\right), \quad \beta_R = \psi\left(\frac{1}{A_R} - 1\right), \quad A_R < 1. \quad (23)$$

The endpoint coefficients thus do double duty: they regularize extrapolation *and* identify the moment critical exponents and the Lee-consistent wing slopes. A calibration may fit L, R freely (subject only to $A_R < 1$) or impose explicit wing priors by constraining A_L, A_R through (17)–(18). The cross-model treatment of wings and Lee bounds is the subject of Note 09; here they fall out of the parametrization for free.

Heuristic

The map ψ is decreasing: a *heavier* tail (larger A , hence smaller critical exponent) gives a *steeper* variance wing (larger β). Equity index smiles have small A_L, A_R — modestly fatter than log-normal — so their wings sit comfortably below the model-free ceiling $\beta = 2$, and a truncation $z \in [-40, 40]$ in double precision is far more than enough (section 5.2).

5 Option pricing and implied-volatility recovery

5.1 The Black map and vega-normalized residuals

The normalized Black call in total-variance units is

$$B(k, w) = \Phi(d_+) - e^k \Phi(d_-), \quad d_{\pm} = -\frac{k}{\sqrt{w}} \pm \frac{\sqrt{w}}{2}, \quad (24)$$

and the implied total variance generated by the slice solves $B(k, w(k, T)) = C_T(k)$. For calibration the loss is expressed in vega-scaled price units rather than raw price, so that it approximates a

volatility error while preserving the exact arbitrage-free price construction during the solve: given quoted volatility σ_i at k_i , with $w_i = \sigma_i^2 T$,

$$r_i(\theta) = \frac{C^{\text{LQD}}(k_i; \theta) - B(k_i, w_i)}{\partial_\sigma B(k_i, w_i) + \eta}, \quad \partial_\sigma B = \varphi(d_+) \sqrt{T}, \quad (25)$$

with a small floor η in the far wings (appendix A, $\eta = \text{_VEGA_FLOOR}$). Working in price space keeps every iterate a valid density; the vega division undoes the price-to-vol scale so the optimizer sees something close to a vol residual.

5.2 Quadrature, Hermite interpolation, and Newton inversion

On a finite logit grid $z_j \in [-Z, Z]$ the pipeline computes, in one pass, $u_j = \Lambda(z_j)$, $h_j = e^{g(u_j)}$, the cumulative $\bar{Q}_j$ by (fourth-order) cumulative Simpson quadrature, the shift μ from (11), and the reverse-cumulative asset share A_j . The two truncation tails are added analytically: at the right,

$$\int_Z^\infty e^Q \Lambda(1 - \Lambda) dz \approx \frac{e^{Q(Z) - Z}}{1 - A_R}, \quad \int_{-\infty}^{-Z} e^Q \Lambda(1 - \Lambda) dz \approx \frac{e^{Q(-Z) - Z}}{1 + A_L}. \quad (26)$$

For equity tails with A_L, A_R small, $Z = 40$ buries the truncation error far below double-precision vega noise.

Two implementation refinements matter for accuracy and for the analytic Jacobian. First, the production build stores the *exact nodal derivatives* $dQ/dz = e^g$ and $dA/dz = -e^Q u(1 - u)$ alongside the nodal values, so both Q and A are interpolated off-grid with *cubic Hermite* polynomials, not linear interpolation. Priced curves are then smooth enough for clean finite-difference Greeks, and — because Hermite interpolation returns the nodal value exactly at a node — calendar constraints expressed at grid z -values match node-indexed ones bit-for-bit. Second, the strike inversion $Q(z_k) = k$ is solved by *Hermite–Newton* iteration to machine precision, not by a coarse table lookup, which keeps the ATM handles (section 6) exact.

Performance

The quadrature grid $(z, u, u(1 - u))$ and the Legendre matrix $P_n(1 - 2u)$ depend only on (Z, n_{pts}) and the model order — never on θ . A single calibration builds the slice many hundreds of times, so these arrays are cached (returned read-only) and only the parameter-dependent combination $g(u)$ is re-formed per call; this alone made `build_slice` about $1.7\times$ faster. Optimization runs on a coarser $n_{\text{pts}} = 2001$ grid whose Simpson error is already orders of magnitude below the fit budget, and the *accepted* slice is rebuilt once at the full grid for reporting, density, var-swap and calendar diagnostics. See appendix B.

6 Exact ATM handles and the orthogonal chart

6.1 Level, skew and curvature from the quantile

Traders want the first coordinates of a model to be *actual* ATM level, skew and curvature. The LQD slice delivers these in closed form, with no finite differencing of implied volatility. Let u_0 solve $Q(u_0) = 0$, set $c_0 = C(0)$, and let $f_0 = f_X(0) = u_0(1 - u_0)e^{-g(u_0)}$ be the ATM return density. The

first two *log-strike* call derivatives are exactly

$$C'(0) = -(1 - u_0), \quad C''(0) = f_0 - (1 - u_0), \quad (27)$$

where C'' is in log-strike (from $C'(k) = -e^k(1 - F_X(k))$). Let w_0 solve the ATM Black identity $B(0, w_0) = 2\Phi(\sqrt{w_0}/2) - 1 = c_0$. With $a = \sqrt{w_0}/2$, $n = \varphi(a)$, $\Phi_a = \Phi(a)$, the Black partials at $k = 0$ are

$$\begin{aligned} B_k &= -(1 - \Phi_a), & B_w &= \frac{n}{2\sqrt{w_0}}, & B_{kk} &= -(1 - \Phi_a) + \frac{n}{\sqrt{w_0}}, \\ B_{kw} &= \frac{n}{4\sqrt{w_0}}, & B_{ww} &= -\frac{n}{4w_0^{3/2}} - \frac{n}{16\sqrt{w_0}}. \end{aligned} \quad (28)$$

Differentiating $B(k, w(k)) = C(k)$ once and twice at $k = 0$ gives

$$w_1 = \frac{C'(0) - B_k}{B_w}, \quad w_2 = \frac{C''(0) - B_{kk} - 2B_{kw}w_1 - B_{ww}w_1^2}{B_w}, \quad (29)$$

and the actual implied-volatility handles are

$$\sigma_0 = \sqrt{\frac{w_0}{T}}, \quad s_0 = \frac{w_1}{2\sqrt{T}w_0}, \quad \kappa_0 = \frac{w_2}{2\sqrt{T}w_0} - \frac{w_1^2}{4\sqrt{T}w_0^{3/2}}. \quad (30)$$

These are exact for every LQD slice. The Vol-Fitter uses them as the carrier of the graph extrapolator (Note 14), which propagates exactly $(\sigma_0, s_0, \kappa_0)$ across the smile graph.

6.2 The ATM-orthogonal chart

Collect the three handles into $H(\theta) = (w_0, s_0, \kappa_0)$ and let $J = \partial H / \partial \theta \in \mathbb{R}^{3 \times d}$ at a reference θ^* (computed by central differences; the quadrature is deterministic, so this is accurate to $\sim 10^{-9}$). The least-norm perturbation realizing a desired handle change δh is $\delta \theta = J^\top (JJ^\top)^{-1} \delta h$, and $\Pi_\perp = I - J^\top (JJ^\top)^{-1} J$ projects onto handle-preserving directions. With $U = J^\top (JJ^\top)^{-1}$ (so $JU = I_3$) and V an orthonormal basis of $\ker J$ from the QR factorization of $\Pi_\perp$, the chart

$$\theta = \theta^* + U\alpha + V\xi \quad (31)$$

has *three primary coordinates* α that move the trader handles and *shape coordinates* ξ that leave them fixed to first order. For larger moves the map is made exact by a three-dimensional Newton solve of $H(\theta^* + U\alpha + V\xi) = h^{\text{target}}$ (the “retarget”); because $JU = I_3$ the 3×3 Jacobian is the identity at the reference, an excellent preconditioner that rarely needs refreshing. This is what lets a trader drag ATM level, skew or curvature while the optimizer keeps using the remaining Legendre modes for wings, shoulders and event convexity.

7 Calibration of one expiry

7.1 The objective

Given quotes $(k_i, \sigma_i, \omega_i)$ at one maturity, with $w_i = T\sigma_i^2$, the calibrator minimizes

$$\min_{\theta} \sum_i \omega_i \left(\frac{C^{\text{LQD}}(k_i; \theta) - B(k_i, w_i)}{\partial_{\sigma} B(k_i, w_i) + \eta} \right)^2 + \lambda \sum_{n \geq 4} n^{2r} a_n^2, \quad (32)$$

as a bound-free nonlinear least-squares problem (scipy `trf`). The high-order ridge $\lambda n^{2r} a_n^2$ is optional and suppresses oscillation in sparse markets; it deliberately leaves the first body modes a_2, a_3 free. The hard admissibility condition is $A_R < 1 - \varepsilon_R$ for a small buffer ε_R . There is *no* density-positivity term — positivity is structural (proposition 1).

The soft tail barrier. To keep the residual smooth as the optimizer approaches the $A_R = 1$ wall, the stacked residual carries a softplus barrier

$$b(\theta) = \log(1 + e^{s(A_R - c)}), \quad (33)$$

with centre $c = 0.90$ and steepness $s = 50.0$ (both `FitSettings` coefficients). The barrier is negligible for $A_R \ll c$ and rises steeply near the wall, pushing the fit back *before* it reaches the hard rejection. When a trial θ does cross $A_R \geq 1$, the price block is replaced by a large smooth-in- A_R penalty $10 + A_R$ so the trust region still has a gradient pointing back to feasibility.

7.2 Initialization

Three robust initializers are available; the default is the *logistic base* $a_n = 0$, $L = R = \log s$ with $s = \sqrt{3w_0}/\pi$ from the variance match $\text{Var}(X) \approx \pi^2 s^2/3$, seeded from the ATM-interpolated quote variance. A *wing-aware base* sets L, R from estimated Lee slopes through (23); a *quantile projection* least-squares-fits g to a trusted external density and is used to warm-start hard event smiles.

7.3 The hooks: bands, var-swaps, and priors

The same objective (32) carries four optional residual blocks that are byte-identical-to-absent when unused, and which connect this note to the rest of the series:

- **Bid-ask / haircut band** (Note 07). The mid residual is replaced by a band objective: the vol band edges become call-price edges, and a one-sided hinge penalizes the model leaving the band plus a small mid anchor, all in the same vega-normalized price space.
- **Variance-swap target** (Note 08). One vol-space penalty pulls the slice's fair var-swap toward a quoted level. LQD prices the var swap by the *exact closed form* $-2\mathbb{E}[X] = -2\int Q(u)du$ (section 7.4), not by replication — so the var-swap fit stays as cheap as the mid fit.
- **Strike-gap prior anchor** (Note 13). Vega-normalized residuals pull the fit toward a transported prior at delta-locations, weighted by how data-starved the smile is.
- **Quote-operator prior** (Note 13). ATM/RR/BF operator residuals pull the model's operators toward the prior's where live quotes do not identify them.

7.4 The var-swap closed form

Because the log contract has the static replication $\mathbb{E}[-2X] = -2 \int_0^1 Q(u)du$, the LQD fair var-swap total variance is a single quadrature against the same grid,

$$w_{\text{vs}} = -2 \int_{-\infty}^{\infty} Q(z) \Lambda(z)(1 - \Lambda(z)) dz, \quad (34)$$

whose integrand decays like $z e^{-|z|}$. This is why the LQD var-swap penalty is free: the generic replication of Note 08 would re-solve implied variance on a strike grid at every Jacobian column and turn a millisecond fit into minutes.

8 The analytic residual Jacobian

The dominant cost of (32) used to be the Jacobian: `scipy` rebuilt the entire quadrature pipeline $P + 1$ times per iteration to finite difference it ($P = d = N + 1$ parameters). The Vol-Fitter instead propagates the exact Jacobian of the whole residual stack in *one* quadrature pass. The key is a cancellation identity, and it is the cleanest piece of mathematics in the implementation.

8.1 The cancellation identity

Regard the priced call (13) as a function of θ through both the slice and the strike root $z_k(\theta)$ that solves $Q(z_k; \theta) = k$:

$$C(k; \theta) = A(z_k; \theta) - e^k (1 - \Lambda(z_k)). \quad (35)$$

Differentiating,

$$\frac{dC}{d\theta} = \underbrace{\frac{\partial A}{\partial \theta} \Big|_z}_{\text{explicit}} + \left[\underbrace{A_z(z_k)}_{=-e^{Q(z_k)} \Lambda(1-\Lambda)} + e^k \Lambda'(z_k) \right] \frac{dz_k}{d\theta}. \quad (36)$$

Now $A_z(z) = -e^{Q(z)} \Lambda(z)(1 - \Lambda(z))$ from (12), and at z_k we have $e^{Q(z_k)} = e^k$ and $\Lambda'(z_k) = \Lambda(1 - \Lambda) = u_k(1 - u_k)$. Hence the bracket is

$$A_z(z_k) + e^k \Lambda'(z_k) = -e^k u_k(1 - u_k) + e^k u_k(1 - u_k) = 0. \quad (37)$$

Theorem 1 (Implicit-root cancellation). *The implicit dependence of the strike root z_k on the parameters drops out of the price gradient, leaving*

$$\boxed{\frac{dC}{d\theta} = \frac{\partial A}{\partial \theta} \Big|_{z=z_k}}. \quad (38)$$

So $dC/d\theta$ is obtained by evaluating the *nodal* parameter-sensitivities of the asset share at the fixed point z_k — by the very same Hermite interpolation used to price, fed the nodal sensitivities $\partial A/\partial \theta_j$ and $\partial A_z/\partial \theta_j$ instead of the nodal values.

8.2 Nodal sensitivities

Everything reduces to differentiating (10)–(12) term by term, using that g is *affine* in θ with constant basis $\phi_j = \partial g / \partial \theta_j$ — namely $\phi_L = 1 - u$, $\phi_R = u$ and $\phi_{a_n} = P_n(1 - 2u)$. Since $\bar{Q}' = e^g$,

$$\frac{\partial \bar{Q}(z)}{\partial \theta_j} = \int_0^z e^{g(\Lambda(s))} \phi_j(\Lambda(s)) \, ds, \quad (39)$$

again by cumulative quadrature, anchored at the centre node. The martingale shift $\mu = -\log M$ with $M = \int e^{\bar{Q}} \Lambda(1 - \Lambda) \, dz$ differentiates to

$$\frac{\partial \mu}{\partial \theta_j} = -\frac{1}{M} \int e^{\bar{Q}} \frac{\partial \bar{Q}}{\partial \theta_j} \Lambda(1 - \Lambda) \, dz + (\text{the two analytic tail-correction terms}), \quad (40)$$

where the tail corrections (26) contribute their own θ -derivatives through $\bar{Q}(\pm Z)$ and through A_L, A_R (whose gradients are $A_L(1, 0, 1 \dots)$ and $A_R(0, 1, (-1)^n \dots)$ from (17)–(18)). Then $\partial Q / \partial \theta_j = \partial \mu / \partial \theta_j + \partial \bar{Q} / \partial \theta_j$, the asset-share sensitivity follows by reverse cumulative quadrature of $\partial(e^Q u(1 - u)) / \partial \theta_j$ plus its right-tail correction, and the nodal $\partial A_z / \partial \theta_j$ is just $-\partial(e^Q u(1 - u)) / \partial \theta_j$. The remaining residual blocks differentiate trivially: the band hinge multiplies $dC/d\theta$ by the violation sign, the ridge block is $\text{diag}(\text{reg})$ on the a -columns, the calendar slack is $\sqrt{\text{cal wt}}(-dA/d\theta)$ on the active constraints, and the barrier row is $\text{expit}(s(A_R - c)) s \partial A_R / \partial \theta$.

8.3 Scope and measured speed-up

The analytic Jacobian covers the configuration with *no* var-swap or prior blocks (the var-swap and prior-anchor terms are not yet differentiated analytically; those fits fall back to `scipy`’s two-point finite difference — correct, merely not accelerated). Table 1 reports a fresh end-to-end comparison: the analytic and finite-difference fits reach the *same* optimum (the costs agree to ~ 11 significant figures, relative difference $\sim 10^{-11}$) while the analytic Jacobian is 1.4–1.7 $\times$ faster across $N = 6 \dots 12$ on this benchmark, with both paths converging in a handful of iterations.

Table 1: Analytic vs. two-point finite-difference Jacobian, fresh run of the production calibrator on the SVI-JW benchmark (fastest of three repeats). $P = N + 1$ is the parameter count; the cost columns confirm an identical optimum.

N	P	analytic (ms)	FD (ms)	speed-up	$ \Delta \text{cost} $
6	7	16.6	22.2	1.34 $\times$	5.6e-20
8	9	19.5	28.7	1.47 $\times$	1.4e-19
10	11	22.7	36.3	1.60 $\times$	8.9e-20
12	13	21.6	36.9	1.71 $\times$	2.0e-20

Heuristic

Why only $\sim 1.5\times$ and not $\sim P\times$? Two reasons. First, the cached static grid makes a *repeat build_slice* cheap, so the $P + 1$ finite-difference builds are not $P + 1$ times the cost of one cold build. Second, `trf` converges here in ~ 7 iterations, so the linear algebra and the single value evaluation dilute the Jacobian saving. The win is larger on heavier configurations and is, importantly, *free of approximation*: the optimum is unchanged. The structural payoff — a

Jacobian that no longer rebuilds the quadrature — is what makes higher-order and surface-wide fits comfortable.

9 Calendar-arbitrage-free surfaces

Proposition 1 kills butterfly arbitrage within a slice but says nothing across expiries. For $T_1 < \dots < T_M$ with each forward-normalized to mean one, absence of calendar arbitrage at fixed log-moneyness is $C_{T_i}(k) \leq C_{T_j}(k)$ for $T_i < T_j$ — equivalently the laws of Y_T are increasing in convex order. With equal means, $Y_{T_i} \leq_{cx} Y_{T_j}$ iff the *integrated upper-quantile* curves are ordered,

$$G_i(\alpha) := \int_{\alpha}^1 e^{Q_i(u)} du \leq G_j(\alpha) \quad \forall \alpha \in [0, 1], \quad (41)$$

with equality at $\alpha = 0$. This is exactly the upper asset share of (12) read at quantile level, so it is already on the grid. The surface is built nearest-to-farthest: each new slice is fitted subject to $G_i(\alpha_m) \geq G_{i-1}(\alpha_m) - \tau_m$ on a dense α -grid, enforced as the soft calendar slack of (32) (penalty weight `calendarWeight`), with the constraint evaluated at z -values so it is exact regardless of the optimization grid. The quantile constraint is better conditioned than a grid of call inequalities because it never inverts Q at a strike and stays meaningful in the wings where IV inversion is ill-conditioned. Note 10 develops the model-agnostic calendar machinery (and the subtlety that the variance floor must be confined to the traded log- k range); here we only record that for LQD the constraint is the native object $A(z)$.

10 Worked examples

All numbers and figures in this section are produced by `Docs/notes/figures/gen_lqd.py`, which fits the production calibrator to two synthetic targets.

10.1 An SPX-like SVI-JW smile

The benchmark target is a six-month SPX-like slice from raw SVI $w(k) = a + b\{\rho(k - m) + \sqrt{(k - m)^2 + \sigma^2}\}$ with $(a, b, \rho, m, \sigma) = (0.010625, 0.07289, -0.5, 0.05831, 0.10100)$, an ATM volatility near 20.6% with a pronounced index put skew. The seven-parameter $N = 6$ fit over $k \in [-0.35, 0.30]$ converges in 7 residual evaluations to the coefficients of table 2, with endpoint and Lee diagnostics

$$A_L = 0.214458, \quad A_R = 0.069023, \quad \mu = 0.020613, \quad \beta_L = 0.097073, \quad \beta_R = 0.035757. \quad (42)$$

For comparison the raw-SVI wing slopes are $b(1 - \rho) = 0.1093$ (left) and $b(1 + \rho) = 0.0364$ (right): the Lee slopes agree to ~ 2 digits on the right wing ($\beta_R = 0.035757$ vs 0.0364) and to ~ 1 on the steeper left ($\beta_L = 0.097073$ vs 0.1093, $\sim 10\%$), even though nothing imposed it — they emerge from the fitted endpoint basis. The exact ATM handles are $\sigma_0 = 20.62\%$, $s_0 = -0.3538$, $\kappa_0 = 1.6474$, and the martingale check returns $\mathbb{E}[e^X] - 1 = 0.00e + 00$. The maximum implied-volatility error over the calibration grid is 0.23 volatility basis points (fig. 1).

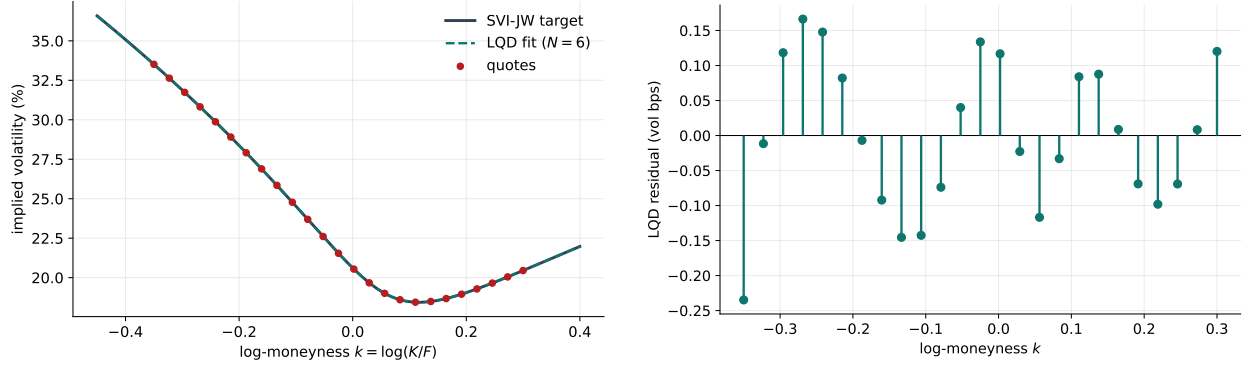


Figure 1: Left: the SVI-JW target, the seven-parameter LQD fit, and the quotes. Right: the LQD implied-volatility residual at the quotes, in volatility basis points. Beyond the calibration window the LQD curve is its own Lee-consistent extrapolation, not an SVI continuation.

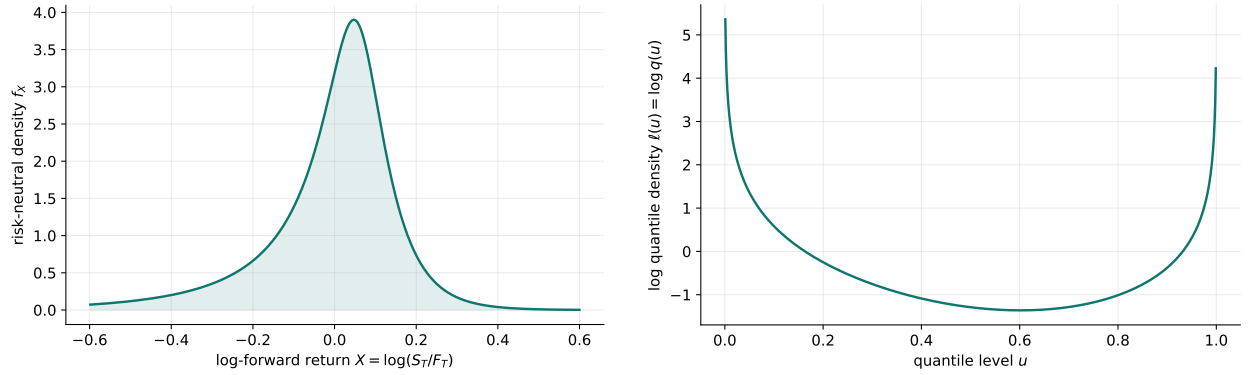


Figure 2: Left: the risk-neutral log-return density implied by the SVI fit — non-negative by construction. Right: the fitted log quantile density $\ell(u) = \log q(u)$; the $-\log u - \log(1-u)$ endpoints are the source of the exponential tails.

Table 2: Seven-parameter LQD fit to the SPX-like SVI-JW slice (fresh run).

Parameter	Value
L	-1.87479110
R	-3.08957252
a_2	+0.38076853
a_3	-0.04700855
a_4	-0.00719617
a_5	+0.00645578
a_6	+0.00213226

10.2 A bimodal “double-hat” event smile

Ahead of a scheduled binary event the terminal density can be *bimodal*, a shape a low-order direct-IV model fights. The target here is an equal mixture of two log-return normals with $\sigma = 5\%$ centred

near $\pm 9\text{--}10\%$ over $T = 30/365$, martingale-normalized. A thirteen-parameter $N = 12$ LQD fit over $k \in [-0.25, 0.25]$ recovers both the smile and the two-humped density (fig. 3), with endpoint scales $A_L = 0.015341$, $A_R = 0.014979$ and a maximum implied-volatility error of 13.19 volatility basis points. Positivity of the density is automatic even where the higher Legendre modes sculpt the two hats; in a sparse market one would lean on the ridge or the ATM-orthogonal chart to avoid spurious oscillation.

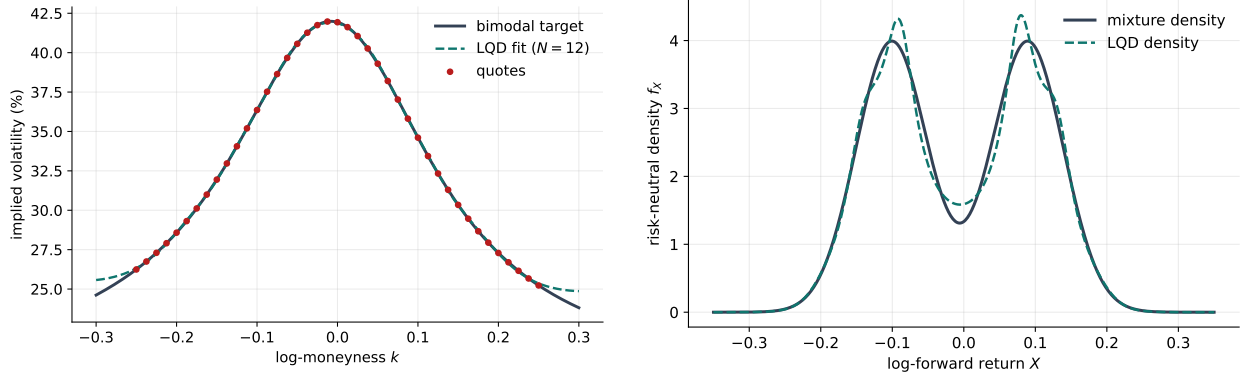


Figure 3: The bimodal event example. Left: target smile and $N = 12$ LQD fit — note the central trough and supported wings that a quadratic-in- k model cannot make. Right: the true mixture density and the LQD density, both with two hats.

11 What is genuinely original here

Quantile-based and density-based smile constructions are not new, and the Lee analysis is classical. Three things in this implementation are worth singling out:

1. **The endpoint basis** — $\log u - \log(1 - u) + (1 - u)L + uR$. It is what makes the tails *exponential with controllable scale* while keeping the body in a clean orthogonal-polynomial space; the Lee slopes (23) then come out of the parametrization rather than being patched on.
2. **The exact ATM handles and the orthogonal chart** (section 6). The level/skew/curvature are closed-form functionals of the quantile, so the model carries genuine trader coordinates *and* a chart in which the optimizer's shape modes do not disturb them — the same three handles that the graph extrapolator propagates in Note 14.
3. **The implicit-root cancellation** (theorem 1). The strike root's dependence on θ vanishing from the price gradient is what turns a $P + 1$ -rebuild finite-difference Jacobian into a one-pass analytic one, with the optimum provably unchanged.

12 Limitations

LQD removes butterfly arbitrage by construction but *not* calendar arbitrage across independently fitted expiries — that needs the convex-order constraints of section 9. Very high Legendre orders can fit noise and sculpt spurious density wiggles; this is a model-risk, not an arbitrage, problem, controlled by the ridge, bid–ask weighting and event priors. Finally the model assumes a continuous

law on $(0, \infty)$: a true atom at zero (jump-to-default) is only *approximated* by a very heavy left tail and would otherwise require an explicit mass point outside the continuous quantile-density component.

13 Traceability

Table 3: Claims in this note and the code/tests that lock them.

Claim	Object	Code anchor	Test anchor
Density positive; call curve monotone and convex; parity holds	proposition 1	models/lqd/ quadrature.py	test_lqd_pricing.py:: test_density_positive, :: test_call_curve_monotone_ and_convex_in_strike, ::test_put_call_parity
Martingale shift correct; $\mathbb{E}[e^X] = 1$ (11)	(11)	models/lqd/ quadrature.py	test_lqd_pricing.py:: test_svi_fit_martingale_ shift_matches_note, ::test_svi_fit_martingale_ integral_is_one
Endpoint scales and Lee slopes; underflow-safe	(17)–(23)	models/lqd/basis.py	test_lqd_basis.py:: test_svi_fit_endpoint_ scales_match_note, ::test_svi_fit_lee_slopes_ match_note
Legendre basis: recursion and orthogonality	(16)	models/lqd/basis.py	test_lqd_basis.py:: test_legendre_recursion_ matches_explicit_ polynomials, ::test_ legendre_orthogonality_ on_unit_interval
Exact ATM handles	(30)	models/lqd/atm.py	test_lqd_atm.py::test_atm_ level_matches_implied_vol, ::test_atm_skew_matches_ finite_difference, ::test_atm_curvature_ matches_finite_difference
Orthogonal chart: shape moves preserve handles; retarget exact	section 6.2	models/lqd/ortho.py	test_lqd_ortho.py:: test_shape_move_leaves_ handles_nearly_unchanged, ::test_retarget_hits_ exact_handles
Analytic Jacobian = FD (mid, band, calendar); same optimum	theorem 1	models/lqd/jacobian. py	test_lqd_jacobian.py:: test_jacobian_matches_fd_ mid, ::test_jacobian_ matches_fd_calendar, ::test_analytic_and_fd_ fits_agree

Claim	Object	Code anchor	Test anchor
Benchmark fit and speed rail	section 10	models/lqd/calibrate.py	test_lqd_calibrate.py:: test_calibrate_to_svi_benchmark, test_calibration_is_fast_enough
Double-hat event smile: bimodal density recovered	section 10	models/lqd/calibrate.py	test_lqd_pricing.py:: test_double_hat_density_is_bimodal
JW conversion used by the benchmark	appendix C	models/svi_jw/svi.py	test_lqd_pricing.py:: test_jw_conversion_matches_note

A Hyperparameter atlas

Every knob touching the LQD slice, with its default and role. Surfaced knobs are user-facing `FitSettings`; hidden knobs are internal constants tuned to the algorithm and not exposed.

Table 4: LQD hyperparameters.

Knob	Default	Role
<i>Surfaced (FitSettings)</i>		
nOrder N	6	Legendre expansion order; 7 parameters at $N = 6$. 6 for ordinary slices, 8–14 for scheduled-event slices.
regLambda λ	10^{-6}	High-order ridge coefficient in (32); suppresses oscillation in sparse data.
regPower r	1.0	Ridge exponent in $n^{2r} a_n^2$; larger r penalizes high modes harder.
barrierCenter c	0.90	Centre of the A_R softplus barrier (33); engages before the $A_R = 1$ wall.
barrierScale s	50.0	Steepness of that barrier.
weightScheme	equal	Per-quote weights ω_i (equal or tv_density); see Note 07.
<i>Hidden (internal constants)</i>		
Z_MAX Z	40.0	Half-width of the symmetric logit quadrature grid; truncation error $\ll$ vega noise for equity tails.
N_POINTS	8001	Reporting/diagnostic grid nodes (odd, so $z = 0$ is a node).
OPT_N_POINTS	2001	Coarser grid used during the optimization iterations; accepted slice rebuilt at N_POINTS.
EPS_AR ε_R	10^{-6}	Buffer in the hard bound $A_R < 1 - \varepsilon_R$.
_VEGA_FLOOR η	10^{-4}	Floor added to Black vega in the residual normalization (25).

Knob	Default	Role
<code>xtol/ftol/gtol</code>	10^{-10}	trf tolerances; ~ 6 orders below the fit budget, so the optimum is display-identical while trf stops grinding invisible reductions.
<code>max_nfev</code>	4000	Evaluation cap (rarely approached).

B Performance notes

1. **Cached static grid.** The grid $(z, u, u(1 - u))$ and the Legendre matrix depend only on (Z, n_{pts}, N) , so they are LRU-cached and returned read-only; only $g(u)$ is re-formed per call. This removed $\sim 40\%$ of `build_slice` and made each build $\sim 1.7\times$ faster.
2. **Two-grid optimization.** Iterations run on `OPT_N_POINTS` = 2001 (Simpson error already far below the fit budget); the accepted slice is rebuilt once at 8001 nodes for the reported result, density, var-swap and calendar diagnostics. Converged parameters agree with a full-grid fit to $\sim 10^{-6}$.
3. **Hermite interpolation/inversion.** Storing the exact nodal derivatives dQ/dz and dA/dz gives cubic-Hermite off-grid evaluation (clean Greeks) and machine-precision strike inversion by Hermite-Newton — and supplies precisely the nodal sensitivities the analytic Jacobian reuses.
4. **Analytic Jacobian** (section 8). One quadrature pass instead of $P + 1$ finite-difference rebuilds, reaching the same optimum (relative cost agreement $\sim 10^{-11}$), measured $1.4\text{--}1.7\times$ end-to-end here and structurally more on heavier configurations. Gated to the var-swap/prior-free configuration.
5. **Looser tolerances.** Dropping **trf** tolerances from 10^{-15} to 10^{-10} stops the optimizer chasing a 10^{-15} cost reduction that is invisible in the priced surface, saving whole $(P + 1)$ -eval iterations at no display-precision cost.

C The SVI-JW conversion used in the example

The SVI-JW parameters $(v, \psi, p, c, \tilde{v})$ relate to raw SVI by $w_0 = vT$, $b = \frac{\sqrt{w_0}}{2}(p + c)$, $\rho = \frac{c-p}{c+p}$, $\psi = \frac{b}{2\sqrt{w_0}}(\rho - \frac{m}{\sqrt{m^2 + \sigma^2}})$ and $\tilde{v}T = a + b\sigma\sqrt{1 - \rho^2}$. Setting $\chi = \frac{m}{\sqrt{m^2 + \sigma^2}} = \rho - \frac{4\psi}{p+c}$ gives $m = \chi\sigma/\sqrt{1 - \chi^2}$ and $\sqrt{m^2 + \sigma^2} = \sigma/\sqrt{1 - \chi^2}$, and subtracting the minimum-variance relation from the ATM relation,

$$\sigma = \frac{w_0 - \tilde{v}T}{b((1 - \rho\chi)/\sqrt{1 - \chi^2} - \sqrt{1 - \rho^2})}, \quad m = \frac{\chi\sigma}{\sqrt{1 - \chi^2}}, \quad a = \tilde{v}T - b\sigma\sqrt{1 - \rho^2}. \quad (43)$$

The benchmark's raw parameters in section 10 are the image, at $T = 0.5$, of

$$(v, \psi, p, c, \tilde{v}) = (0.0425, -0.25, 0.75, 0.25, 0.034).$$

D Reference implementation

The whole slice is one quadrature pass in the logit coordinate $z = \log(u/(1-u))$: form g , integrate $dQ/dz = e^g$ to the quantile, solve one scalar normalization for the martingale shift μ , and reverse-integrate the asset-share $A(z)$; every strike is then priced by $C(k) = A(z_k) - e^k(1-u_k)$. The listing below reproduces `build_slice`'s quantile, asset share and μ to 10^{-8} on the $N=6$ benchmark and gives $\mathbb{E}[e^X] = 1.000000$ (production inverts $Q(z_k) = k$ and interpolates A with cubic Hermite for smooth Greeks; the linear `np.interp` here agrees to 10^{-7} on the dense grid).

Listing 2: The LQD quadrature pipeline (distilled from `models/lqd/{basis,quadrature}.py`).

```
1 def legendre(n_max, x):                                     # P_0...P_{n_max}, 3-term
    recursion
2     P = np.empty((n_max+1, x.size)); P[0] = 1.0; P[1] = x
3     for n in range(1, n_max):
4         P[n+1] = ((2*n+1)*x*P[n] - n*P[n-1]) / (n+1)
5     return P
6
7 def g_of_u(u, L, R, a):                                     # smooth part g(u) of ell(
    u)
8     return (1-u)*L + u*R + a @ legendre(len(a)+1, 1 - 2*u)[2:]
9
10 def build_slice(z, L, R, a):                                 # one quadrature pass, eqs
    (q_logit)-(mu_norm)
11     u = expit(z); dz = z[1] - z[0]
12     A_L, A_R = np.exp(g_of_u(np.array([0.0, 1.0]), L, R, a)) # tail
    scales e^{g(0)}, e^{g(1)}
13     Q = cumulative_simpson(np.exp(g_of_u(u, L, R, a)), dx=dz, initial
    =0.0) # dQ/dz = e^{g}
14     Q -= Q[len(z)//2]                                       # anchor Q(0) = 0
15     mass = np.exp(Q)*u*(1-u)
16     tot = np.trapezoid(mass, z) + np.exp(Q[-1]-z[-1])/(1-A_R) + np.exp
    (Q[0]-z[-1])/(1+A_L)
17     mu = -np.log(tot); Q = Q + mu                           # martingale normalization
    : E[e^X] = 1
18     m = np.exp(Q)*u*(1-u)
19     A = cumulative_simpson(m[:, -1], dx=dz, initial=0.0)[:, -1] + np.exp
    (Q[-1]-z[-1])/(1-A_R)
20     return u, Q, A                                           # asset share A(z) = int_z
    ^inf e^{Q} u(1-u)
21
22 def call_price(k, z, u, Q, A):                               # C(k) = A(z_k) - e^k (1 -
    u_k)
23     zk = np.interp(k, Q, z)                                  # invert Q(z_k) = k
24     return np.interp(zk, z, A) - np.exp(k)*(1 - expit(zk))
```

References

- [1] D. Breeden and R. Litzenberger. Prices of state-contingent claims implicit in option prices. *Journal of Business*, 51(4):621–651, 1978.

- [2] R. W. Lee. The moment formula for implied volatility at extreme strikes. *Mathematical Finance*, 14(3):469–480, 2004.
- [3] J. Gatheral. *The Volatility Surface: A Practitioner’s Guide*. Wiley, 2006.
- [4] J. Gatheral and A. Jacquier. Arbitrage-free SVI volatility surfaces. *Quantitative Finance*, 14(1):59–71, 2014.
- [5] V. Strassen. The existence of probability measures with given marginals. *Annals of Mathematical Statistics*, 36(2):423–439, 1965.
- [6] P. Carr and D. Madan. Towards a theory of volatility trading. In *Volatility*, Risk Books, 1998.